Basic (your first coroutine) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `coroutine_basics_practice.py`: "`async def` doesn't run like a normal function" is the single fact every later exercise in this document assumes you already internalized — see it fail once, on purpose, here, before it costs you a confusing bug in real code. **If not:** the first time you'd meet an unawaited coroutine would be a silent `RuntimeWarning` buried in real output, with no smaller example to recognize it from.

Solution

```
# coroutine_basics_practice.py
import asyncio


async def wait_and_return():
    await asyncio.sleep(1)
    return "done"


# Calling a coroutine function doesn't run it — you get a coroutine object.
unstarted = wait_and_return()
print(unstarted)   # <coroutine object wait_and_return at 0x...>

# asyncio.run() actually runs it and hands back the result.
result = asyncio.run(wait_and_return())
print(result)       # "done"
```

Expected output:

```
<coroutine object wait_and_return at 0x7f2a1c0b1a40>
done
sys:1: RuntimeWarning: coroutine 'wait_and_return' was never awaited
```